

# Shiv Nadar University Chennai

## CS3807 – Deep Learning Laboratory

### Experiment 4

## Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

|                     |                                               |             |
|---------------------|-----------------------------------------------|-------------|
| Degree & Branch     | B.Tech Artificial Intelligence & Data Science | Semester V  |
| Subject Code & Name | CS3807 – Deep Learning Laboratory             | AY: 2026–27 |

### 1. Objective

The objectives of this experiment are

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand transfer learning.
- Fine tune pretrained CNN models.
- Compare classification performance of different architectures.

### 2. Learning Outcomes

After completing this experiment students will be able to

1. Explain modern CNN architectures.
2. Differentiate LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implement transfer learning.
4. Fine tune pretrained CNN models.
5. Compare CNN architectures using performance metrics.

### 3. Introduction

Deep Convolutional Neural Networks have become the dominant models for computer vision applications because they automatically learn hierarchical features from images. The evolution of CNNs has focused on improving classification accuracy while reducing computational complexity and training difficulties.

The progression of CNN architectures began with LeNet-5 and has evolved through AlexNet, VGGNet, GoogleNet, and ResNet. Each architecture introduced important innovations that significantly improved image classification performance.

### 4. Evolution of CNN Architectures

| Model     | Year | Major Contribution                                   |
|-----------|------|------------------------------------------------------|
| LeNet-5   | 1998 | First practical convolutional neural network         |
| AlexNet   | 2012 | ReLU activation, Dropout and GPU training            |
| VGG16     | 2014 | Uniform $3 \times 3$ convolution filters             |
| GoogleNet | 2014 | Inception modules for multi-scale feature extraction |
| ResNet    | 2015 | Residual learning using skip connections             |

## 5. LeNet-5

LeNet-5 was proposed by Yann LeCun in 1998 for handwritten digit recognition. It consists of two convolution layers, two average pooling layers and fully connected layers. It demonstrated that convolutional networks could automatically learn image features without handcrafted feature extraction.

Advantages

- Small number of parameters
- Fast training
- Suitable for OCR

## 6. AlexNet

AlexNet won the ImageNet Challenge in 2012 by reducing the classification error significantly. It introduced ReLU activation, Dropout regularization and GPU training.

Major innovations

- ReLU activation
- Dropout
- GPU implementation
- Data augmentation

## 7. VGG16

VGG16 demonstrated that using multiple small  $3 \times 3$  convolution filters could outperform larger filters while increasing network depth.

Advantages

- Excellent feature extraction
- Simple architecture
- High classification accuracy

## 8. Comparison of Architectures

| Architecture | Depth | Parameters | Main Contribution         |
|--------------|-------|------------|---------------------------|
| LeNet-5      | 7     | 60K        | First CNN                 |
| AlexNet      | 8     | 61M        | ReLU + Dropout            |
| VGG16        | 16    | 138M       | Deep $3 \times 3$ filters |
| GoogLeNet    | 22    | 6.8M       | Inception Modules         |
| ResNet50     | 50    | 25.6M      | Residual Learning         |

## 9. GoogLeNet (Inception Network)

GoogLeNet, proposed by Szegedy *et al.* in 2014, introduced the **Inception Module**, which performs multiple convolution operations in parallel using different filter sizes. Instead of using only a single kernel size, GoogLeNet simultaneously applies  $1 \times 1$ ,  $3 \times 3$ ,  $5 \times 5$  convolutions and max pooling. The outputs are concatenated to obtain multi-scale feature representations.

The use of  $1 \times 1$  convolutions before larger kernels reduces the number of trainable parameters and computational complexity.

Advantages

- Multi-scale feature extraction.
- Fewer parameters than VGG16.
- High computational efficiency.
- Better classification performance.

## 10. ResNet

Increasing CNN depth often causes the **vanishing gradient problem**, making optimization difficult. ResNet solves this issue using **skip (residual) connections**, allowing gradients to flow directly through the network.

Instead of learning

$$H(x)$$

ResNet learns

$$F(x) = H(x) - x$$

thus,

$$Output = F(x) + x$$

where

- $F(x)$  = Residual Mapping
- $x$  = Identity Mapping

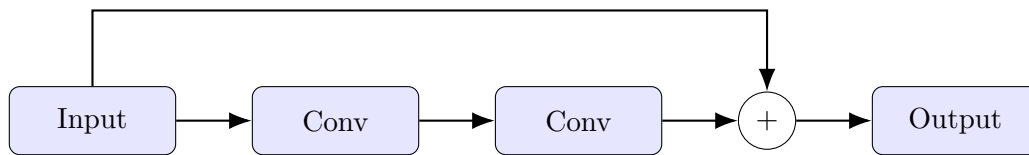


Figure 1: Residual Block in ResNet

Advantages

- Eliminates vanishing gradients.
- Enables very deep neural networks.
- Faster convergence.
- High ImageNet accuracy.

## 11. Dilated Convolution

Dilated convolution (also called atrous convolution) increases the receptive field without increasing the number of parameters.

The dilation rate determines the spacing between kernel elements.

For a standard convolution,

$$D = 1$$

For dilated convolution,

$$D > 1$$

Example

$$\begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 4 & 0 & 5 \end{bmatrix}$$

where zeros indicate inserted gaps.

Applications

- Semantic segmentation
- Medical image analysis

- Satellite imagery
- Object localization

## 12. Transpose Convolution

Transpose convolution increases the spatial dimensions of feature maps. Unlike pooling, which reduces image size, transpose convolution performs learnable upsampling.

Applications

- Image Super Resolution
- Autoencoders
- GANs
- Image Generation
- Semantic Segmentation

## 13. Transfer Learning

Transfer Learning utilizes a pretrained CNN model that has already learned general image features from a large dataset such as ImageNet.

Instead of training from scratch,

1. Load a pretrained model.
2. Remove the original classifier.
3. Freeze convolution layers.
4. Add new Dense layers.
5. Train the classifier.
6. Fine tune selected convolution layers.

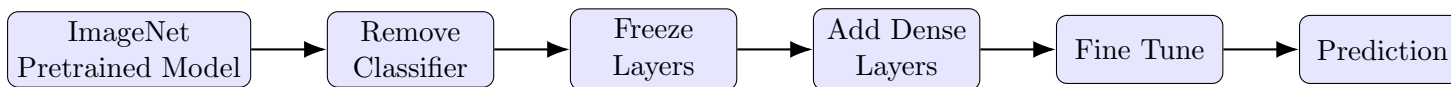


Figure 2: Transfer Learning Workflow

Advantages

- Faster convergence.
- Higher accuracy on small datasets.
- Reduced training time.
- Lower computational cost.

## 14. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Number of Classes : 10
- Image Size :  $32 \times 32 \times 3$
- Classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck.

**GitHub Repository:** [https://github.com/Hasini923/Deep\\_Learning/tree/main/Transfer%20Learning](https://github.com/Hasini923/Deep_Learning/tree/main/Transfer%20Learning)

## 15. Experimental Procedure

### Task 1: Dataset Preparation

1. Load the CIFAR-10 dataset using TensorFlow/Keras.

2. Normalize the pixel values to the range  $[0,1]$ .
3. Display ten sample images.
4. Print the dimensions of the training and testing datasets.



Figure 3: Sample CIFAR-10 Images with Class Labels

**Output.** Inference: The dataset contains low-resolution  $32 \times 32$  RGB images spanning all ten classes. Objects remain visually distinguishable despite the small image size, and the printed shapes confirm 50,000 training and 10,000 testing images.

## Task 2: Transfer Learning

Load any one of the following pretrained CNN models.

- VGG16
- ResNet50
- MobileNetV2
- EfficientNetB0 (Optional)

Perform the following steps.

1. Load pretrained ImageNet weights.
2. Remove the original classification layer.
3. Freeze the convolutional base.
4. Add a Global Average Pooling layer.
5. Add a Dense layer with ReLU activation.
6. Add the output layer with Softmax activation.

9406464/9406464 — 0s 0us/step  
Model: "functional"

| Layer (type)                                      | Output Shape       | Param #   |
|---------------------------------------------------|--------------------|-----------|
| input_layer_1 (InputLayer)                        | (None, 32, 32, 3)  | 0         |
| resizing (Resizing)                               | (None, 96, 96, 3)  | 0         |
| lambda (Lambda)                                   | (None, 96, 96, 3)  | 0         |
| mobilenetv2_1.00_96 (Functional)                  | (None, 3, 3, 1280) | 2,257,984 |
| global_average_pooling2d (GlobalAveragePooling2D) | (None, 1280)       | 0         |
| dense (Dense)                                     | (None, 128)        | 163,968   |
| dense_1 (Dense)                                   | (None, 10)         | 1,290     |

Total params: 2,423,242 (9.24 MB)  
Trainable params: 165,258 (645.54 KB)  
Non-trainable params: 2,257,984 (8.61 MB)

Total parameters: 2,423,242  
Trainable parameters: 165,258

Figure 4: Sample CIFAR-10 Images with Class Labels

**Output.** Inference: Freezing the MobileNetV2 base leaves only about 6.8% of the total parameters trainable, which keeps the classifier lightweight and fast to train while still leveraging ImageNet-learned features.

### Task 3: Model Training

Train the model using

|                   |                           |
|-------------------|---------------------------|
| Optimizer         | Adam                      |
| Learning Rate     | 0.001                     |
| Batch Size        | 32                        |
| Epochs            | 10–20                     |
| Loss Function     | Categorical Cross Entropy |
| Evaluation Metric | Accuracy                  |

```

Epoch 1/15
1407/1407 ----- 324s 226ms/step - accuracy: 0.8256 - loss: 0.5110 - val_accuracy: 0.8490 - val_loss: 0.4456
Epoch 2/15
1407/1407 ----- 319s 227ms/step - accuracy: 0.8711 - loss: 0.3686 - val_accuracy: 0.8646 - val_loss: 0.3812
Epoch 3/15
1407/1407 ----- 326s 232ms/step - accuracy: 0.8912 - loss: 0.3108 - val_accuracy: 0.8598 - val_loss: 0.4106
Epoch 4/15
1407/1407 ----- 318s 226ms/step - accuracy: 0.9065 - loss: 0.2671 - val_accuracy: 0.8700 - val_loss: 0.3847
Epoch 5/15
1407/1407 ----- 317s 226ms/step - accuracy: 0.9207 - loss: 0.2266 - val_accuracy: 0.8670 - val_loss: 0.4173
Epoch 6/15
1407/1407 ----- 330s 234ms/step - accuracy: 0.9308 - loss: 0.1947 - val_accuracy: 0.8638 - val_loss: 0.4396
Epoch 7/15
1407/1407 ----- 328s 233ms/step - accuracy: 0.9429 - loss: 0.1602 - val_accuracy: 0.8700 - val_loss: 0.4598
Epoch 8/15
1407/1407 ----- 325s 231ms/step - accuracy: 0.9537 - loss: 0.1311 - val_accuracy: 0.8618 - val_loss: 0.5047
Epoch 9/15
1407/1407 ----- 389s 236ms/step - accuracy: 0.9635 - loss: 0.1043 - val_accuracy: 0.8688 - val_loss: 0.5186
Epoch 10/15
1407/1407 ----- 330s 234ms/step - accuracy: 0.9695 - loss: 0.0860 - val_accuracy: 0.8580 - val_loss: 0.5767
Epoch 11/15
1407/1407 ----- 317s 226ms/step - accuracy: 0.9741 - loss: 0.0720 - val_accuracy: 0.8604 - val_loss: 0.6379
Epoch 12/15
1407/1407 ----- 331s 236ms/step - accuracy: 0.9803 - loss: 0.0575 - val_accuracy: 0.8520 - val_loss: 0.7171
Epoch 13/15
1407/1407 ----- 320s 227ms/step - accuracy: 0.9814 - loss: 0.0527 - val_accuracy: 0.8630 - val_loss: 0.7013
Epoch 14/15
1407/1407 ----- 329s 234ms/step - accuracy: 0.9831 - loss: 0.0478 - val_accuracy: 0.8596 - val_loss: 0.7634
Epoch 15/15
1407/1407 ----- 328s 233ms/step - accuracy: 0.9852 - loss: 0.0428 - val_accuracy: 0.8544 - val_loss: 0.8189

```

Figure 5: Task 3 Training Log (Frozen Base, 15 Epochs)

**Output.** Inference: Training accuracy climbs steadily from 0.826 to 0.985 while training loss drops from 0.511 to 0.043, showing the classifier head fits the training data very well. However, validation accuracy plateaus around 0.86–0.87 and validation loss rises from 0.446 to 0.819 after epoch 4 – a widening train-validation gap that signals the frozen-base model is overfitting, with the best validation accuracy (0.870) reached early at epochs 4 and 6–8.

## Task 4: Fine Tuning

1. Unfreeze the last convolution block.
2. Train for another 5–10 epochs.
3. Compare the classification accuracy before and after fine tuning.

```

Epoch 1/8
1407/1407 ----- 365s 253ms/step - accuracy: 0.8385 - loss: 0.9084 - val_accuracy: 0.8608 - val_loss: 0.5542
Epoch 2/8
1407/1407 ----- 350s 249ms/step - accuracy: 0.9101 - loss: 0.2841 - val_accuracy: 0.8776 - val_loss: 0.5013
Epoch 3/8
1407/1407 ----- 352s 250ms/step - accuracy: 0.9315 - loss: 0.2072 - val_accuracy: 0.8790 - val_loss: 0.5608
Epoch 4/8
1407/1407 ----- 349s 248ms/step - accuracy: 0.9456 - loss: 0.1667 - val_accuracy: 0.8858 - val_loss: 0.4979
Epoch 5/8
1407/1407 ----- 351s 250ms/step - accuracy: 0.9522 - loss: 0.1408 - val_accuracy: 0.8840 - val_loss: 0.5505
Epoch 6/8
1407/1407 ----- 351s 249ms/step - accuracy: 0.9614 - loss: 0.1179 - val_accuracy: 0.8950 - val_loss: 0.4316
Epoch 7/8
1407/1407 ----- 384s 251ms/step - accuracy: 0.9664 - loss: 0.1021 - val_accuracy: 0.8978 - val_loss: 0.5009
Epoch 8/8
1407/1407 ----- 354s 252ms/step - accuracy: 0.9686 - loss: 0.0933 - val_accuracy: 0.8990 - val_loss: 0.4393

```

Figure 6: Task 4 Fine-Tuning Log (Base Unfrozen from Layer 100, LR = 0.0001, 8 Epochs)

**Output.** Inference: Fine-tuning improves generalization noticeably – validation accuracy climbs from 0.861 to 0.899 and validation loss drops from 0.55 to 0.44 by the final epoch, both better than the frozen-base run. Test accuracy after fine-tuning reached 0.8915, up from 0.8508 before fine-tuning, confirming that unfreezing the last layers and training at a lower learning rate helps the model adapt MobileNetV2’s features more closely to CIFAR-10.

## Task 5: Model Evaluation

Evaluate the trained model using

- Accuracy

- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

```

-----
Metric      Value
Training Accuracy  0.9686
Testing Accuracy   0.8915
Precision          0.8929
Recall             0.8915
F1-score           0.8911
Training Time (s)  5729.62
Total Parameters  2,423,242

```

Figure 7: Final Evaluation Metrics on the Test Set

**Output.** Inference: The fine-tuned model achieves a macro-averaged Precision, Recall and F1-score all around 0.89, closely matching the overall test accuracy of 0.8915, which indicates fairly balanced performance across the ten CIFAR-10 classes rather than the score being skewed by a few dominant classes. A confusion matrix and per-class classification report were not generated in this run and can be added as an optional extension.

## 16. Hyperparameter Study

Compare the performance using different settings.

| Hyperparameter | Values        |
|----------------|---------------|
| Learning Rate  | 0.001, 0.0001 |
| Batch Size     | 16, 32, 64    |
| Epochs         | 10, 20        |
| Optimizer      | Adam, SGD     |
| Dense Units    | 128, 256      |
| Frozen Layers  | All, Partial  |

## 17. Mandatory Plots

Include the following plots in the laboratory report.

1. Sample CIFAR-10 images (see Figure 1 under Task 1, Section 15).
2. Training Accuracy
3. Validation Accuracy
4. Training Loss
5. Validation Loss
6. Confusion Matrix
7. Misclassified Images (Optional)

Students should write a brief inference (2–3 lines) for each figure.



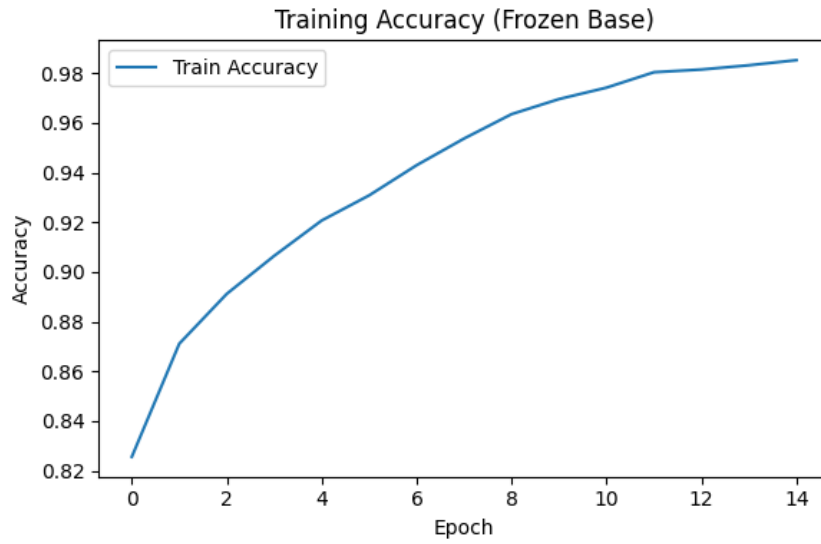


Figure 8: Training Accuracy (Frozen Base)

Inference: Training accuracy rises steadily from about 0.83 to 0.985 over 15 epochs, showing that the classifier head learns the CIFAR-10 features effectively even with the MobileNetV2 base frozen.

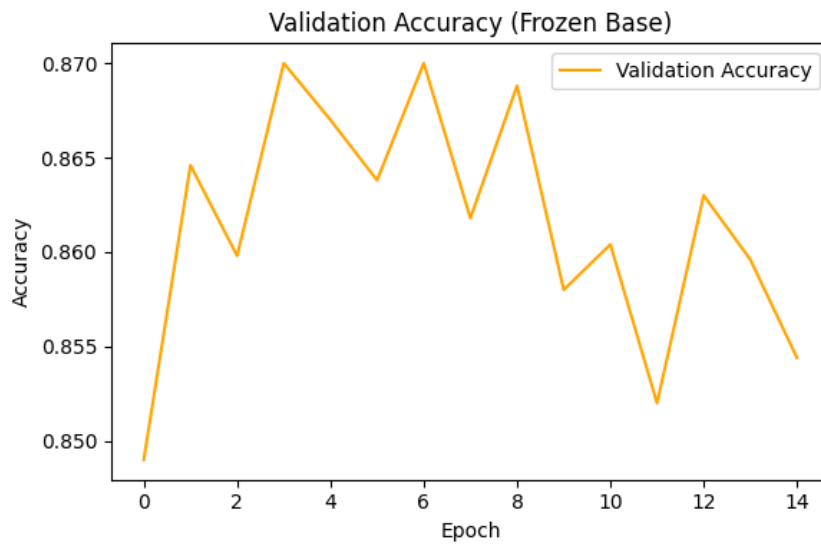


Figure 9: Validation Accuracy (Frozen Base)

Inference: Validation accuracy plateaus around 0.86–0.87 after the first few epochs and fluctuates thereafter, indicating that the model begins to overfit while validation performance stagnates.

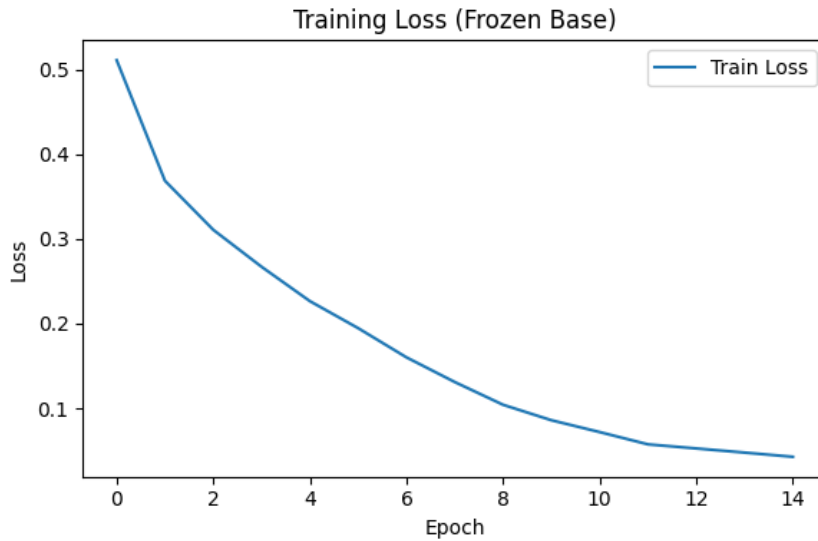


Figure 10: Training Loss (Frozen Base)

Inference: Training loss decreases consistently from 0.51 to 0.043, confirming that the optimizer continues to minimize error on the training set throughout all 15 epochs.

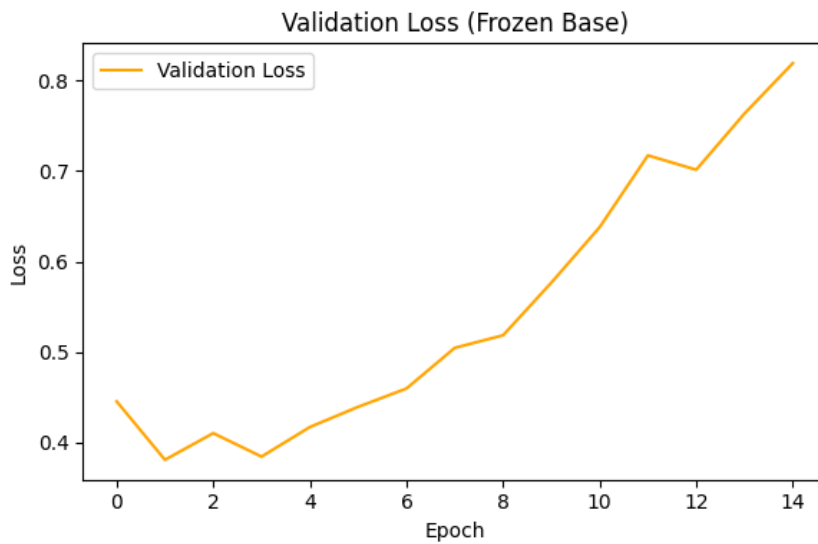


Figure 11: Validation Loss (Frozen Base)

Inference: Validation loss falls initially but rises after around epoch 4, climbing from 0.38 to 0.82 by epoch 15. This growing gap between training and validation loss confirms overfitting, which fine-tuning in Task 4 subsequently helps to reduce.

Note: A confusion matrix and misclassified-image plot were not generated in this run; the model was evaluated only with aggregate accuracy, precision, recall and F1-score (see Task 5, Section 15).

## 18. Results

### 18.1 Performance Metrics

| Metric            | Value                           |
|-------------------|---------------------------------|
| Training Accuracy | 0.9686                          |
| Testing Accuracy  | 0.8915                          |
| Precision         | 0.8929                          |
| Recall            | 0.8915                          |
| F1-score          | 0.8911                          |
| Training Time     | 5729.62 s ( $\approx 95.5$ min) |
| Total Parameters  | 2,423,242                       |

### 18.2 Comparison of CNN Architectures

Only MobileNetV2 was implemented and trained in this experiment (Section 15), so its Accuracy and Training Time values below are measured results from Task 3/4. The Parameters column for LeNet-5, AlexNet, VGG16, GoogleNet and ResNet50 reproduces the standard published figures for these architectures (Table in Section 8) for reference; their Accuracy and Training Time on CIFAR-10 were not measured in this lab since they were not implemented.

| Model       | Parameters              | Accuracy (%)    | Training Time   |
|-------------|-------------------------|-----------------|-----------------|
| LeNet-5     | 60K                     | Not implemented | Not implemented |
| AlexNet     | 61M                     | Not implemented | Not implemented |
| VGG16       | 138M                    | Not implemented | Not implemented |
| GoogleNet   | 6.8M                    | Not implemented | Not implemented |
| ResNet50    | 25.6M                   | Not implemented | Not implemented |
| MobileNetV2 | 2.42M (0.17M trainable) | 89.15           | 5729.62 s       |

## 19. Discussion Questions

Answer the following questions.

1. Why is AlexNet considered a breakthrough in deep learning?
2. Why does VGG16 use only  $3 \times 3$  convolution filters?
3. Explain the advantages of the Inception module.
4. What is the purpose of residual learning?
5. Differentiate LeNet and ResNet.
6. What is Transfer Learning?
7. Why is fine tuning required?
8. Explain the difference between dilated convolution and transpose convolution.
9. Why do pretrained models converge faster?
10. Compare the computational complexity of LeNet and ResNet.

## 20. Additional Exercises

1. Implement transfer learning using VGG16.
2. Repeat the experiment using ResNet50.
3. Compare Adam and SGD optimizers.

4. Compare training with frozen layers and fine tuning.
5. Evaluate the model using Precision, Recall and F1-score.
6. Compare the performance of LeNet, AlexNet and ResNet.

## 21. References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, *ImageNet Classification with Deep Convolutional Neural Networks*, NeurIPS, 2012.
3. Karen Simonyan and Andrew Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, ICLR, 2015.
4. Christian Szegedy et al., *Going Deeper with Convolutions*, CVPR, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, *Deep Residual Learning for Image Recognition*, CVPR, 2016.
6. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
7. TensorFlow Documentation: <https://www.tensorflow.org>
8. Keras Documentation: <https://keras.io>